data

MDPI

Data Descriptor

A Pilot-Real-Calibrated Indoor Robotic IoT Benchmark Dataset for Edge-Assisted Mobile Robot Navigation and Anomaly Detection

Burak Aggul

Department of Database, Network Design and Management, Plato Vocational School, Istanbul Topkapi University, Istanbul 34020, Türkiye; burakaggul@topkapi.edu.tr

Abstract

Mobile robots used in edge-assisted Industrial Internet-of-Things (IIoT) settings generate coupled motion, LiDAR, edge-compute, and network telemetry. Public datasets that place these streams in one tabular format, with scenario labels suitable for machine-learning experiments, are still limited. This data descriptor presents a pilot-real-calibrated indoor robotic IoT benchmark dataset with 120,000 records sampled at 2 Hz across nominal navigation and nine anomaly scenarios. The benchmark rows are generated from physically constrained simulation rules and are explicitly labeled as synthetic benchmark data. Real pilot evidence is included separately: ROS Noetic runs on a TurtleBot3 Burger, successful LD08 LiDAR bringup after resolving a driver mismatch, and NVIDIA Jetson Nano tegrastats logs under normal-navigation workloads. The calibrated file aligns normal-navigation LiDAR and edge-compute distributions with these pilot measurements while keeping the multi-scenario structure needed for controlled anomaly-detection experiments. The package includes CSV files, metadata, a data dictionary, validation reports, baseline scripts, ROS collection utilities, and a plan for future fully physical data collection. The complete dataset is openly available on Zenodo.

Dataset: https://doi.org/10.5281/zenodo.20346694

Dataset License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Keywords: robotic IoT; TurtleBot3; Robot Operating System (ROS); LD08 LiDAR; Jetson Nano; edge computing; anomaly detection; distribution calibration; reproducible datasets; data descriptor

Check for updates

Academic Editor: Joaquín Torres-Sospedra

Received: 4 June 2026
Revised: 25 June 2026
Accepted: 30 June 2026
Published: 4 July 2026

Copyright: © 2026 by the author. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Mobile robots are increasingly used in smart factories, warehouses, and laboratory-scale industrial testbeds, where motion control, sensing, edge processing, and wireless communication are tightly coupled. ROS and ROS 2 have become common middleware layers for collecting and coordinating these robotic streams [1–4]. Edge, AIoT, cloud-robotics, and mobile-edge computing architectures make it possible to process telemetry close to the robot rather than sending every signal to a remote cloud service [5–8]. In these environments, navigation safety depends not only on the mechanical state of the platform but also on LiDAR quality, local compute load, battery behavior, timing jitter, packet loss, and the synchronization of heterogeneous logs.

Small disruptions can therefore become system-level events. A temporary CPU spike can delay local feature extraction; a short wireless outage can change the timing of status messages; a noisy scan can obscure an obstacle-like pattern in the robot's surroundings. This setting connects the present benchmark to the wider anomaly-detection literature, including recent surveys, deep and shallow detection methods, AIoT anomaly detection, robot anomaly datasets, ROS-oriented intrusion datasets, and edge-based robotic-arm monitoring systems [5,9–18]. Machine-learning models can help study these patterns, but they need datasets in which robot state, LiDAR summaries, edge-compute telemetry, network indicators, and scenario labels are available together.

The practical difficulty is that fully physical multi-scenario data collection is expensive, slow, and sometimes unsafe when the scenarios include close obstacles, emergency stops, low battery, packet loss, and wheel slip. Recent public robotic datasets often emphasize raw sensor recordings, SLAM benchmarking, visual place recognition, mobile-robot visual anomalies, robot machine-data anomalies, self-driving laboratory anomalies, or ROS/ROS 2 cyber-physical intrusion traces [4,12–14,19–22]. Many edge-IoT anomaly experiments, however, need an aligned tabular file that can be loaded directly into baseline classifiers and regression models. Table 1 compares the submitted benchmark with representative public datasets and explains this positioning. The dataset described here addresses this gap as a calibrated benchmark, not as a claim of complete physical recording coverage.

Table 1. Comparison between the submitted benchmark and representative public robotic/anomaly datasets.

Dataset	Primary Modality	Scale and Origin	Availability	Positioning Relative to This Dataset
voraus-AD [12]	Industrial robot machine data from a pick-and-place arm	2122 time-series samples and 130 signals at 500 Hz; physical recordings with induced anomalies	Public benchmark	Strong robot-arm anomaly benchmark, but not a mobile robotic-IoT table combining LiDAR summaries, edge-compute metrics, network telemetry, and navigation scenario labels.
ROSIDS23 [14]	ROS-oriented network and intrusion-detection traces	ROS cyber-security traffic records reported in the source dataset; ROS-focused intrusion-detection testbed	Public dataset	Focuses on cyber-intrusion detection for ROS traffic rather than mobile robot navigation kinematics, LiDAR summaries, edge telemetry, and operational anomaly scenarios.
Hazards& Robots [13]	Image-based robot hazard and visual anomaly data	More than 200,000 labelled visual frames across robot exploration datasets; physical robot recordings	Public dataset	Well suited to visual hazard detection, but it does not provide an aligned tabular representation of robot state, edge-compute load, and network metrics.
Hilti SLAM and Hilti-Oxford [19,20]	Raw SLAM sensor logs including cameras, LiDAR, IMU, and ground-truth trajectories	Multi-sequence raw sensor benchmarks with millimeter-level ground truth; physical indoor/outdoor recordings	Public challenge datasets	Optimized for SLAM, localization, and sensor-fusion benchmarking rather than labelled edge-assisted anomaly detection in tabular robotic-IoT streams.
This dataset [23]	Tabular mobile robot kinematics, LiDAR summaries, edge metrics, network metrics, and scenario labels	120,000 rows, 38 columns, and 10 balanced scenarios; physically constrained synthetic benchmark calibrated with TurtleBot3/LD08/Jetson pilot evidence	Zenodo DOI; CC BY 4.0	Provides an accessible CSV benchmark for reproducible edge-assisted mobile robot anomaly-detection prototyping.

To address this need, this paper presents a pilot-real-calibrated indoor robotic IoT benchmark dataset for prototyping edge anomaly-detection pipelines. The main contributions are:

1. We detail a hardware-in-the-loop testbed consisting of TurtleBot3 Burger robots, an NVIDIA Jetson Nano Edge AI node, Raspberry Pi telemetry loggers, and an i9 Master workstation.
2. We present a multi-scenario benchmark dataset populated by physically constrained simulation rules, containing 10 scenarios (one nominal baseline and nine anomaly conditions), 100 scenario runs, two robot streams per run, and 120,000 tabular entries.
3. We detail a calibration procedure using physical pilot trials, including ROS Noetic execution on the TurtleBot3, LD08 LiDAR driver resolution, and CPU/RAM/GPU/temperature logging on the Jetson Nano Developer Kit.
4. We provide open-source logging, parsing, feature-engineering, and validation scripts, together with a staged plan for fuller real-world replication.

In accordance with the FAIR guiding principles for reproducible data sharing [24], this paper avoids over-claiming the dataset's physical recording coverage. The real pilot files are provided as calibration and provenance evidence, while the full multi-scenario benchmark remains a calibrated synthetic dataset. This distinction allows researchers to use the benchmark for controlled prototype development while understanding which components were physically observed and which were statistically generated.

The remainder of this article is organized as follows. Section 2 describes the dataset structure and file hierarchy. Section 3 reports the physical pilot collection, synthetic generation, and calibration steps. Section 4 describes the rows, columns, and data dictionary. Section 5 presents technical validation and baseline ML results. Section 6 outlines possible downstream uses, followed by the limitations in Section 7, the future real-world collection plan in Section 8, and the conclusions in Section 9.

2. Dataset Description

The dataset package is organized for direct loading, feature inspection, and statistical modeling. It contains subfolders for processed synthetic components, calibrated benchmark outputs, pilot evidence, metadata, reports, notebooks, figures, and ROS collection utilities. The overall file structure is shown in Figure 1.

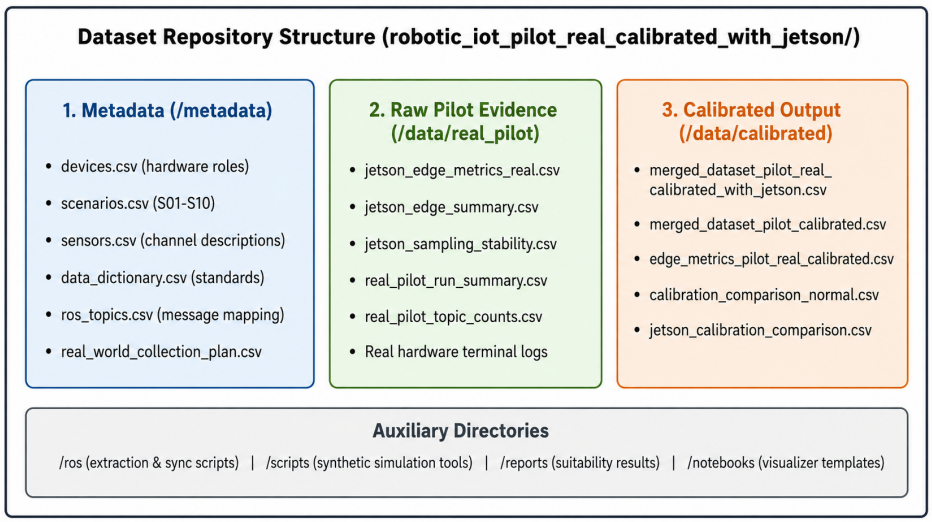


Figure 1. Folder structure of the submitted Zenodo research repository, showing the relationship between metadata files, raw pilot evidence, calibrated outputs, and auxiliary code/report directories.

Figure 1 separates the repository into three evidence layers. The metadata layer defines the devices, scenarios, sensors, data dictionary, ROS topics, and future collection plan needed to interpret the tabular records. The raw pilot evidence layer contains the physical

ROS and Jetson Nano files used as empirical grounding, including topic counts, pilot run summaries, Jetson logs, and terminal evidence. The calibrated output layer contains the final benchmark CSV files created after applying metadata constraints and pilot-derived calibration statistics. The auxiliary directories contain scripts, reports, and notebooks for reproducing validation checks and extending the collection workflow.

The primary data record is `merged_dataset_pilot_real_calibrated_with_jetson.csv` in the `data/calibrated/` subfolder. It contains 120,000 rows and 38 columns covering movement, LiDAR summaries, edge-compute metrics, communication indicators, labels, and calibration provenance. The file is a pilot-real-calibrated synthetic benchmark: normal-navigation LiDAR and edge-compute distributions are aligned with physical pilot evidence, while the full multi-scenario benchmark is generated from controlled rules. The Zenodo release is provided as a citable dataset record [23]. Table 2 lists the key files included in the submitted Zenodo package, together with their roles, row counts, and approximate disk footprints.

Table 2. Key data, metadata, and validation files included in the submitted Zenodo dataset repository.

File Path	Category	Rows	Size	Description & Role
`data/calibrated/merged_..._jetson.csv`	Calibrated Core	120,000	40 MB	Final pilot-real-calibrated synthetic benchmark. Normal-navigation LiDAR and edge features are aligned with pilot measurements.
`data/calibrated/merged_..._calibrated.csv`	Calibrated	120,000	40 MB	Benchmark dataset aligned with ROS Noetic LD08 LiDAR pilot distances.
`data/processed/merged_dataset.csv`	Synthetic	120,000	30 MB	Combined multi-system parameters produced from statistical rules.
`data/processed/robot_state.csv`	Synthetic	120,000	16 MB	Kinematic parameters, velocities, yaw, battery, and slip variables.
`data/processed/lidar_features.csv`	Synthetic	120,000	12 MB	Minimum, mean, and standard-deviation scan distance features.
`data/processed/edge_metrics.csv`	Synthetic	120,000	13 MB	Computing loads, local RAM usage, performance metrics, and processing times.
`data/processed/network_metrics.csv`	Synthetic	120,000	12 MB	Wi-Fi propagation latency, jitter, and packet loss percentage.
`data/processed/scenario_labels.csv`	Synthetic	120,000	8.7 MB	Class flags, run index, severity labels, and anomaly indicators.
`data/real_pilot/jetson_edge_metrics_real.csv`	Real Pilot	61	9.9 KB	Real-world `tegrastats` system logging on NVIDIA Jetson Nano.
`data/real_pilot/real_pilot_run_summary.csv`	Real Pilot	3	1.4 KB	Raw rosbag recording information, duration, file size, and notes.
`data/real_pilot/real_pilot_topic_counts.csv`	Real Pilot	18	1.6 KB	Verified ROS topic message frequencies and recording counts.
`metadata/data_dictionary.csv`	Metadata	38	5.8 KB	Data types, ranges, physical units, and missing-value policies.

The `metadata/` folder contains the files used to interpret the dataset:

1. `devices.csv` outlines the components of our integrated robotic-edge testbed and their data contributions.
2. `scenarios.csv` establishes the standard scenario IDs, names, and injected bounds.
3. `sensors.csv` catalogs the virtual sensing mechanisms and logging rates.
4. `data_dictionary.csv` acts as the master dictionary, outlining parameter limits, units, types, and missing-value interpolation rules.
5. `ros_topics.csv` details the ROS message types and expected frequencies.
6. `real_world_collection_plan.csv` records the future hardware-deployment steps.

The `ros/` folder contains the scripts and launch templates used for pilot collection and planned physical data expansion. It includes launch configurations, edge-monitoring services, feature-extraction scripts, and timestamp synchronizers, so that the pilot setup and the replication pathway can be inspected.

3. Methods

The pilot-real-calibrated dataset was assembled through four connected steps: hardware-in-the-loop (HIL) design, physical pilot runs, scenario-specific synthetic generation, and statistical distribution calibration. The section begins with the ten benchmark scenarios and their temporal structure (Section 3.1), then describes the physics-constrained generation rules used to create the tabular benchmark (Section 3.2). The hardware testbed is introduced in Section 3.3. Sections 3.4 and 3.5 report the ROS/LD08 and Jetson Nano pilot evidence used to ground the benchmark. Section 3.6 then explains how the synthetic distributions were calibrated against the physical pilot measurements. Figure 2 outlines these processing steps.

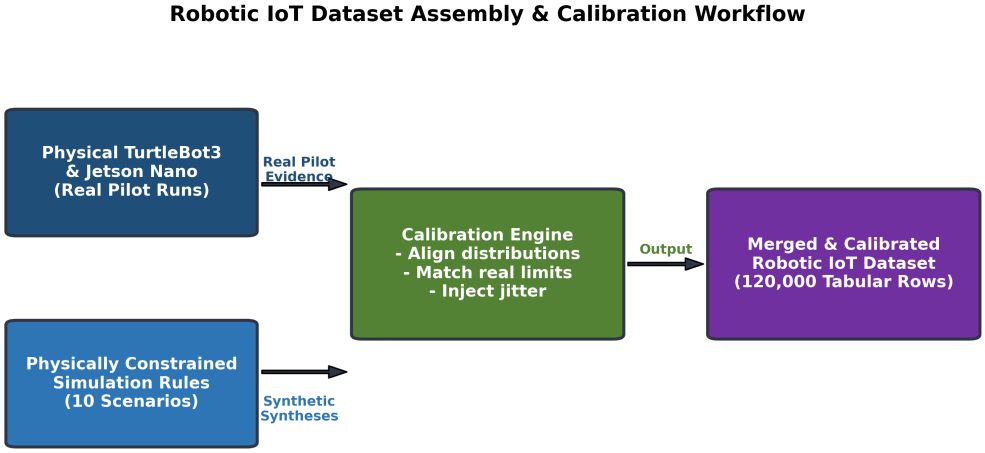


Figure 2. Dataset construction workflow linking physical pilot runs, scenario-specific synthetic generation, statistical calibration, and final validation outputs.

Figure 2 summarizes the assembly process. Physical pilot runs provide empirical LiDAR and edge-compute reference distributions, while the synthetic generation module creates controlled scenario coverage across nominal and anomalous conditions. The calibration engine shifts and scales selected normal-navigation variables against pilot statistics. The final dataset is then checked through range validation, calibration comparison, and baseline ML experiments. This workflow separates the physically observed components from the controlled benchmark syntheses.

3.1. Benchmark Scenario Design

We defined ten scenarios covering normal navigation and common failure conditions in smart industrial environments. The anomaly scenarios represent physical platform issues, sensing errors, compute spikes, and wireless communication bottlenecks that can occur in AGV deployments. Table 3 lists the scenario IDs, names, technical details, default severity levels, and binary classifications.

Figure 3 illustrates the TurtleBot run plan used to organize the benchmark rows. The left panel is a schematic indoor route plan for the two robot streams, not an exact metric floor map. The right panel shows the temporal structure: each run lasts 300 s, scenario S01 remains nominal throughout, and scenarios S02–S10 retain a 0–30 s nominal warm-up before the anomaly condition becomes active from 30–300 s.

For each scenario, we generated ten five-minute navigation runs. Each run includes two TurtleBot3 robot streams. At 2 Hz, each robot stream contributes 600 records per run; therefore, each scenario contributes $10 \times 2 \times 600 = 12{,}000$ rows, and the full 10-scenario benchmark contains 120,000 rows. In anomalous runs (*S*02–*S*10), the failure is not introduced at the start. The first 30 s remain nominal to represent initialization. The selected

anomaly is active from $t = 30$ s to $t = 300$ s, with its intensity scaled by the configured scenario severity parameter. This structure provides transition boundaries for temporal anomaly classification.

Table 3. The 10 robotic IoT research scenarios containing nominal operations and physical/virtual anomalies.

ID	Scenario Name	Technical Description	Severity	Label
S01	`normal_navigation`	Baseline nominal indoor navigation. No active anomalies.	0.00	0 (Normal)
S02	`obstacle_near`	Static or dynamic obstacle appears extremely close to the path.	0.45	1 (Anomaly)
S03	`network_delay`	High communications latency and packet jitter over the Wi-Fi.	0.55	1 (Anomaly)
S04	`packet_loss`	Intermittent router connectivity issues causing high packet loss.	0.60	1 (Anomaly)
S05	`cpu_stress`	Heavy auxiliary background CPU stress on the processor core.	0.65	1 (Anomaly)
S06	`lidar_noise`	Saturated and highly fluctuating LiDAR laser measurement signals.	0.50	1 (Anomaly)
S07	`multi_robot_interference`	Cross-signals and network congestion from multi-robot operational overlap.	0.58	1 (Anomaly)
S08	`emergency_stop`	Abrupt safety shutdown triggered (sudden, unplanned command zeroing).	0.90	1 (Anomaly)
S09	`low_battery`	Rapid battery voltage drop near the lower operational boundary (<10 V).	0.70	1 (Anomaly)
S10	`wheel_slip`	Friction loss and traction slippage causing high odometry-command drift.	0.75	1 (Anomaly)

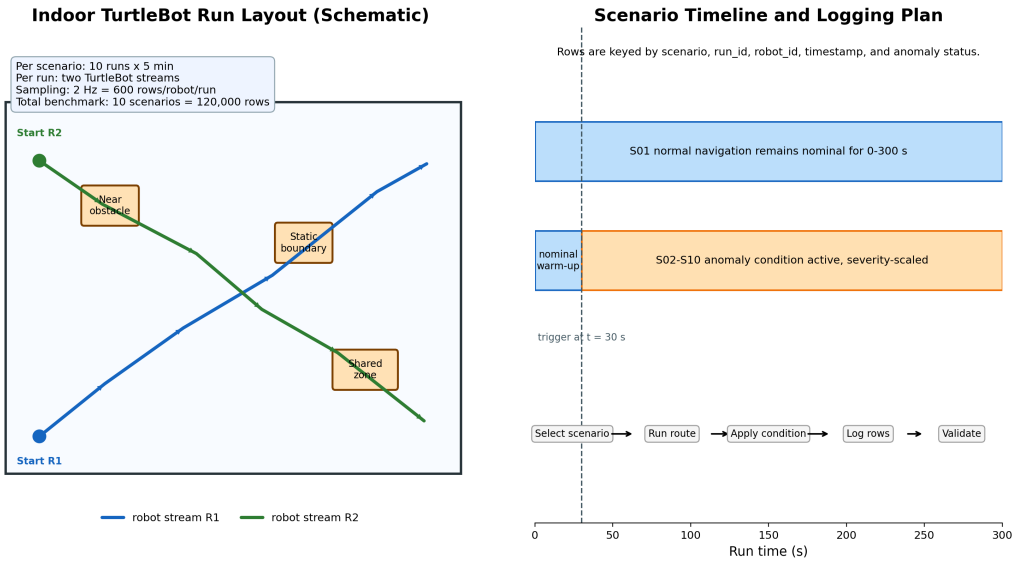


Figure 3. Schematic TurtleBot benchmark run plan. The left panel illustrates two robot streams following planned indoor paths with potential obstacle, boundary, and shared-zone interactions. The right panel shows the 300 s run timeline, including the 0–30 s nominal warm-up and the 30–300 s active anomaly interval for S02–S10.

3.2. Synthetic Data Generation Logic

The synthetic baseline data (`merged_dataset.csv`) were generated with physics-constrained equations for mobile base kinematics, laser reflections, communication metrics, and computing loads. The submitted benchmark stores engineered tabular features rather than full SLAM trajectories, but its kinematic and LiDAR assumptions follow common practice in mobile-robot localization, LiDAR odometry, multi-sensor SLAM, indoor mapping, and SLAM benchmarking [20,25–29]. The kinematic variables are governed by standard differential-drive motion models. The commanded linear velocity v_c and angular velocity

ω_c are represented by sinusoidal variations bounded by the TurtleBot3 Burger hardware specifications ($v_{\max} = 0.22\,\mathrm{m/s}$, $\omega_{\max} = 2.84\,\mathrm{rad/s}$) [30]. The odometry-estimated velocities (v_o, ω_o) are modeled as:

$$v_o(t) = v_c(t) \cdot (1 - \sigma_{\text{slip}}) + \eta_v(t) \quad (1)$$

$$\omega_o(t) = \omega_c(t) + \eta_\omega(t) \quad (2)$$

where σ_{slip} represents the active wheel slip ratio (close to 0 normally, but rising during scenario S10), and $\eta(t)$ represents the measurement noise. The robot coordinates $(x(t), y(t))$ are computed by integrating the velocities over time, and the heading $\theta(t)$ is modeled as a wrapped random walk within the interval $[-\pi, \pi]$ rad.

The laser distance variables $(d_{\min}, d_{\text{mean}}, d_{\text{std}})$ represent reflections from static walls and obstacles. For normal navigation, we model a standard indoor clearance distance using a truncated normal distribution. Under obstacle-near conditions (S02), $d_{\min}$ drops below 0.3 m, and obstacle density increases.

The IoT network variables model ping-response delays. The baseline Wi-Fi round-trip latency L_{net} is modeled using a positive-skew Gamma distribution representing nominal wireless transmission:

$$L_{\text{net}} \sim \text{Gamma}(\alpha = 2.0, \beta = 5.0) + c_{\text{base}} \quad (3)$$

Under scenario S03 (network delay), we simulate channel congestion by shifting the scale parameter and increasing the baseline offset c_{base}. Packet loss percentage is modeled using a beta distribution, which remains low in nominal runs but rises during scenario S04.

The edge-compute variables model local system resources. The nominal CPU load (C_{cpu}) and RAM utilization (R_{ram}) use truncated normal distributions to represent standard runtime behavior. In scenario S05 (CPU stress), CPU utilization is shifted toward higher ranges using a step trigger to simulate heavy active workloads on the edge node.

3.3. Physical Testbed and Pilot Implementation

To ground the benchmark in physical hardware, we used a small pilot testbed made up of robotic-IoT and computing nodes. The system architecture and data-flow pathways are shown in Figure 4.

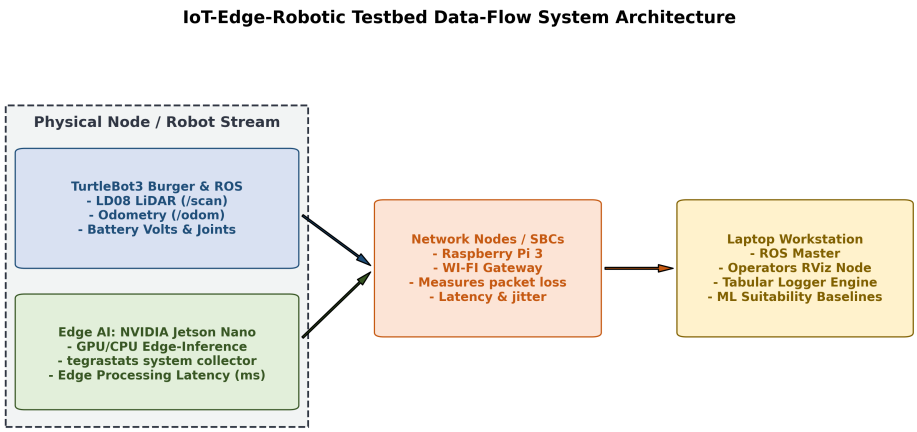


Figure 4. Integrated robotic-IoT testbed architecture showing the TurtleBot3 robot node, ROS communication streams, edge-compute node, network logging devices, and central workstation used for collection and validation.

Figure 4 shows the division of responsibilities in the physical pilot setup. The TurtleBot3 platform publishes robot-state and LiDAR-related ROS topics, the Jetson Nano provides edge-compute telemetry through `tegrastats`, Raspberry Pi devices support network and auxiliary logging, and the workstation coordinates storage, parsing, calibration, and ML validation. This separation is the reason the submitted dataset includes both raw pilot evidence files and calibrated analytical outputs.

The devices used in this experimental testbed are detailed in Table 4.

Table 4. Hardware devices, experimental testbed roles, configurations, and data contributions.

Device ID	Type	Model	Operational Role	Hardware Config	Data Contribution
`tb3_burger_01`	Robot	TurtleBot3 Burger	Mobile robot stream 1	Raspberry Pi 3B+, OpenCR	ROS state and LiDAR scan telemetry
`tb3_burger_02`	Robot	TurtleBot3 Burger	Mobile robot stream 2	Raspberry Pi 3B+, OpenCR	ROS state and LiDAR scan telemetry
`jetson_nano_01`	Edge Node	NVIDIA Jetson Nano	GPU edge inference node	Quad-core ARM, 4 GB RAM	CPU, RAM, GPU, temperature, and latency
`rpi3_01`	IoT Monitor	Raspberry Pi 3B	Wi-Fi latency monitor	Quad-core ARM, 1 GB RAM	Network latency, jitter, and packet loss
`rpi3_02`	IoT Monitor	Raspberry Pi 3B	Auxiliary sync node	Quad-core ARM, 1 GB RAM	Synchronization drift measurements
`laptop_i9_01`	Workstation	Intel i9 Laptop	ROS Master and SQLite logger	8-core CPU, 32 GB RAM	Consolidated tabular records
`laptop_i5_01`	Operator	Intel i5 Laptop	Teleop and RViz control	Quad-core i5, 16 GB RAM	Operator `cmd_vel` messages

The mobile base consists of a TurtleBot3 Burger robot equipped with an OpenCR microcontroller board, a Raspberry Pi 3B+ single-board computer (SBC), and an onboard LiDAR unit for 360-degree range scans. The OpenCR microcontroller manages low-level motor PID controls, wheel joint encoders, and inertial measurement unit (IMU) telemetry, communicating directly with the Raspberry Pi SBC over a serial bus. The Pi SBC runs Ubuntu Mate 20.04 and ROS Noetic, acting as the local robot node and publishing kinematics and range scans over a local 2.4 GHz Wi-Fi router.

The edge-processing workspace incorporates an NVIDIA Jetson Nano Developer Kit (4GB RAM) acting as a local edge node [31]. In the pilot runs reported here, the Jetson Nano was used to record hardware performance under lightweight normal-navigation workloads. A Python-based utility queried system parameters through the `tegrastats` monitoring tool and stored CPU usage, RAM usage, GPU load, temperature, and edge-processing time in structured CSV format.

3.4. ROS Pilot Trials and Driver Resolution

We conducted physical pilot runs to capture empirical baseline levels and verify the ROS launch configuration on the robot. During these runs, we encountered and resolved a LiDAR driver issue.

The TurtleBot3 Burger was initially configured with the default ROS package `hls_lfcd_lds_driver` to interface with the onboard LiDAR. However, the initial pilot runs (`SMOKE_TEST_001` and `SMOKE_TEST_LIDAR_001`) did not publish laser scans on the `/scan` topic. Terminal trace review indicated a protocol mismatch: the LiDAR unit on this robot was an LD08 model, which required a different driver path.

To resolve this, we built the `ld08_driver` ROS package and updated the launch parameters to target `/dev/ttyUSB0` at 230,400 baud, with the frame ID mapped to `laser`. A subsequent pilot run (`SMOKE_TEST_LIDAR_OK_001`) verified that 360-degree laser range scans were being collected. Table 5 summarizes the ROS pilot bag recordings and driver bringup results.

Table 5. Real-world ROS Noetic pilot bag recordings, execution parameters, and driver bringup findings.

Run ID	Scenario	Duration	Size	Messages	ROS Status	LiDAR Driver Bringup Findings
SMOKE_TEST_001	normal_nav	26.8 s	280.8 KB	6442	Running	Kinematic streams active. LiDAR missing (/scan: 0 msgs) due to hls_lfcd_lds_driver protocol incompatibility.
SMOKE_TEST_LIDAR_001	normal_nav	26.8 s	252.5 KB	6322	Running	Retried after hard reset. LiDAR still failed to publish scans under the standard package driver.
SMOKE_TEST_LIDAR_OK_001	normal_nav	27.4 s	86.0 KB	214	Running	Success. Range sensor (/scan) recorded at 7.81 Hz using custom-built ld08_driver and frame laser.

3.5. Jetson Nano Edge Pilot Runs

We also ran edge-computing pilot trials on the NVIDIA Jetson Nano node to capture system performance logs. These trials were configured under normal-navigation conditions, with the Jetson Nano running a lightweight Python 3.8 script that parsed real-time hardware statistics from `tegrastats`.

These pilot runs provide empirical baseline levels for the edge-compute metrics. Table 6 summarizes CPU utilization, RAM usage, GPU load, device temperature, and processing time.

Table 6. Statistical summary of real NVIDIA Jetson Nano hardware metrics during edge pilot runs.

Run ID	Samples	Duration	CPU (%)	RAM (%)	GPU (%)	Temp (°C)	Latency (ms)	Workload Type
JETSON_NORMAL_001	49	59.39 s	5.87 ± 4.15	31.59 ± 0.12	0.00	50.0 ± 0.0	64.71 ± 7.21	Jetson pilot log
JETSON_SMOKE_LIGHT_001	12	14.32 s	4.21 ± 1.35	31.51 ± 0.00	0.00	50.0 ± 0.0	43.11 ± 2.73	Minimal logger baseline

The Jetson pilot runs reveal key physical metrics:

1. **Low Nominal CPU Load:** Under normal-navigation workloads, CPU utilization in the main Jetson calibration run averaged $5.87 \pm 4.15\%$. This indicates that the telemetry logger introduced a light workload.
2. **Stable RAM Footprint:** Locally consumed RAM remained steady at $31.59 \pm 0.12\%$ of the 4GB onboard memory.
3. **GPU Usage at Zero:** GPU load registered at 0% because the pilot logging scripts did not run CUDA-accelerated neural networks.
4. **Thermal Stability:** Onboard temperature sensors reported a stable temperature of 50.00 °C.
5. **Edge Processing Time:** Edge processing time averaged 64.71 ± 7.21 ms, reflecting natural scheduling jitter in the lightweight logging workflow.

3.6. Distribution Calibration Procedure

We used a calibration pipeline to align selected benchmark distributions with the physical pilot measurements (Figure 5). This design follows the practical motivation behind recent sim-to-real and hardware-in-the-loop validation work: simulation is useful for controlled coverage, but physical measurements are still needed to expose calibration limits and transfer assumptions [32,33].

Figure 5 shows the calibration logic used after synthetic scenario generation. For each calibrated feature, the pipeline reads the synthetic benchmark distribution and the corresponding pilot reference statistics, applies mean–variance scaling, clips the result to the configured physical bounds, and writes calibration-status fields into the output table. Calibration was applied to nominal LiDAR and edge-compute ranges, while anomaly scenarios retained their controlled scenario rules.

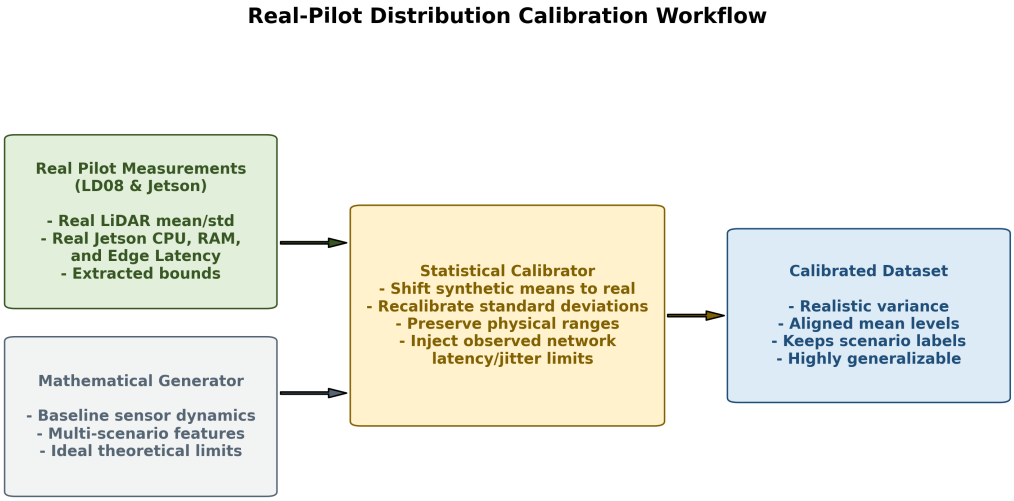


Figure 5. Statistical distribution calibration workflow showing how pilot-derived mean, variance, and clipping constraints are applied to selected synthetic benchmark variables.

The calibration engine processes the generated synthetic data and adjusts target feature distributions to match the mean, variance, and observed margins from the physical pilot runs. This adjustment is achieved using statistical scaling and shifting:

$$x_{calibrated}(t) = \mu_{real} + \left(\frac{\sigma_{real}}{\sigma_{synth}}\right) \cdot (x_{synth}(t) - \mu_{synth}) \quad (4)$$

where μ_{real} and σ_{real} are the mean and standard deviation from our physical trials. If the target column is bounded (such as local CPU usage bounded by $[0, 100]\%$, or distance bounded by positive numbers), the outputs are clipped to these physical ranges:

$$x_{final}(t) = \max(\text{lower_limit},\ \min(\text{upper_limit},\ x_{calibrated}(t))) \quad (5)$$

In the final calibrated dataset (`merged_dataset_pilot_real_calibrated_with_jetson.csv`), normal-navigation LiDAR summaries are aligned with the LD08 pilot feature statistics, and normal-navigation edge metrics are aligned with the empirical distributions from the `JETSON_NORMAL_001` trial. The anomaly scenarios retain their synthetic scenario rules and labels. In this way, the dataset keeps controlled multi-scenario coverage while anchoring nominal operating ranges to pilot hardware measurements. Recent edge outlier-detection and embedded fault-monitoring work further supports the use of lightweight local processing for streaming telemetry and irregular sensor signals [18,34].

4. Data Records

The primary calibrated dataset consists of 120,000 tabular rows, where each row represents a system state observation at a specific timestamp. The columns capture synchronized features across four telemetry categories: robot state, LiDAR metrics, edge-compute states, and network performance. The file also includes provenance fields such as `data_origin`, `calibration_status`, and `calibration_reference`. Table 7 summarizes these categories, along with their key columns, physical units, and hardware sources.

Table 7. Main feature classes, integrated column headers, physical units, and hardware source references.

Variable Category	Key Column Names	Physical Units	Analytical Source
Robot Kinematics & State	`cmd_linear_velocity_mps`, `odom_linear_velocity_mps`, `odom_x_m`, `odom_y_m`, `yaw_rad`, `battery_voltage_v`, `cmd_odom_slip_ratio`, `emergency_stop_event`	$m\,s^{-1}$, $rad\,s^{-1}$, m, V, ratio	TurtleBot3 odometry, IMU, and OpenCR board
LiDAR Features	`min_lidar_distance_m`, `mean_lidar_distance_m`, `lidar_distance_std_m`, `obstacle_density`, `lidar_outlier_rate`	m, ratio, %	360-degree LiDAR range scans (`/scan`)
Edge Metrics	`cpu_usage_percent`, `ram_usage_percent`, `gpu_usage_percent`, `device_temperature_c`, `edge_processing_time_ms`	%, °C, ms	Jetson Nano developer kit `tegrastats` logs
Network Performance	`network_latency_ms`, `jitter_ms`, `packet_loss_percent`	ms, %	Wi-Fi router ping and socket monitors
Benchmark Labels	`run_id`, `scenario_id`, `scenario_name`, `anomaly_label`, `anomaly_severity`	alphanumeric, 0/1 flag, severity fraction	Centralized execution coordinator logger

The schema and validation constraints for each category are summarized below:

1. **Kinematics and Robot State:** Columns such as `cmd_linear_velocity_mps` and `odom_linear_velocity_mps` store velocities in meters per second ($m\,s^{-1}$). Values are constrained by target hardware limits, including a maximum linear velocity of $0.22\,m\,s^{-1}$ and an angular velocity range of $\pm 2.84\,rad\,s^{-1}$. Robot positions (`odom_x_m`, `odom_y_m`) track the integrated odometry trajectory. Onboard battery voltage (`battery_voltage_v`) starts at a fully charged level of ≈12.2 V and decays over time, dropping rapidly (<10.0 V) during low-battery runs (*S*09). The command-odometry slip ratio (`cmd_odom_slip_ratio`) is normally close to 0, but rises in wheel-slip scenarios (*S*10) to represent loss of traction:

$$\text{slip_ratio} = \frac{|v_c - v_o|}{v_c + \epsilon} \tag{6}$$

2. **LiDAR Scanner Features:** Columns like `min_lidar_distance_m` and `mean_lidar_distance_m` capture distances in meters (m) within the configured sensor bounds. The `obstacle_density` feature represents the proportion of beams that detect objects within a near-field clearance boundary (<1.0 m). The `lidar_outlier_rate` field represents the proportion of outlier-like scan readings and rises during LiDAR noise runs (*S*06) to represent signal degradation, such as lens dust or sensor faults.
3. **Edge Computing Metrics:** These columns represent computing loads on the edge node. CPU utilization (`cpu_usage_percent`) and system RAM consumption (`ram_usage_percent`) are expressed as percentages. Local temperatures are tracked in degrees Celsius (`device_temperature_c`). Local edge processing times (`edge_processing_time_ms`) are logged in milliseconds.
4. **Network Performance Metrics:** These columns track Wi-Fi network telemetry. Wireless propagation delays (`network_latency_ms`) and jitter variations (`jitter_ms`) are measured in milliseconds. The packet loss field (`packet_loss_percent`) tracks the percentage of dropped packets, which rises during communications failures (*S*04).

5. Technical Validation

We checked the dataset against several data-quality and modeling-readiness criteria. Table 8 summarizes the validation checklist.

Table 8. Summary of technical validation domains, verification strategies, and final evaluation status.

Validation Domain	Verification & Evaluation Strategy	Status/Result
Required Fields	Verified all required analytical fields across 120,000 rows. The optional `calibration_reference` field is intentionally populated only for calibrated normal-navigation rows.	Complete required schema
Range and Limits	Validated against boundary limits for velocities, voltage, LiDAR summaries, temperature, network delay, and packet loss.	Verified in limits
Timestamp Alignment	Confirmed parseable ISO 8601-1:2019 [35] timestamps, monotonic order within each run–robot stream, and a 2 Hz sampling interval.	Uniform monotonic
LiDAR Verification	Normal-navigation LiDAR features aligned to empirical LD08 pilot summary statistics.	Calibrated output
Edge Metric Profile	Normal-navigation edge columns aligned with real NVIDIA Jetson Nano `tegrastats` logs.	Calibrated edge columns
Scenario Balance	Confirmed 12,000 rows for each of the ten scenario labels.	Balanced classes
ML Task Baseline	Run-based Random Forest validation achieved binary F1 = 1.000 and ten-class accuracy = 0.9286.	Suitable for benchmarking

5.1. Data Quality and Range Validation

We validated data quality by identifying missing fields, checking range boundaries, and verifying timestamp monotonicity. The final calibrated file contains 120,000 rows, 38 columns, 100 run identifiers, two robot streams, and 12,000 rows per scenario. Required analytical fields contain no missing values. The optional `calibration_reference` field is populated for the 12,000 normal-navigation rows calibrated against pilot evidence and left blank for scenarios without direct pilot calibration references. Feature distributions were checked against the configured limits of the hardware testbed:

- Velocity measurements were verified within safe bounds ($0 \leq v_{odom} \leq 0.23\,\mathrm{m/s}$, $-2.85 \leq \omega_{odom} \leq 2.85\,\mathrm{rad/s}$).
- Battery voltage was checked to ensure it remained within the physical operational limits of the battery pack (9.8 V to 12.4 V).
- Compute temperature values were verified within configured operational bounds (20.0 °C to 95.0 °C).

Timestamp parsing was verified using mixed ISO 8601-1:2019 formats, and every run–robot stream was monotonic at the expected 0.5 s interval.

5.2. Calibration Quality Assessment

We evaluated the calibration pipeline by comparing calibrated benchmark values with the physical pilot runs under normal navigation. Figures 6 and 7 compare selected calibrated distributions across scenarios.

The distribution comparisons are useful in two ways. First, Figure 6 shows that the `cpu_stress` scenario has a higher CPU usage distribution than the nominal and most non-compute scenarios, indicating that the edge-compute channel carries the intended high-load signal. Some anomalies remain close to the nominal CPU range because their primary effects are expected in other channels, such as LiDAR, network, battery, or command–odometry consistency. Second, Figure 7 shows that spatial scenarios reduce or alter the minimum-distance distribution, especially where near-obstacle or shared-space effects are encoded. These plots should therefore be read as channel-specific validation figures, not as evidence that every anomaly is separable in every feature. Overall, the calibration pipeline:

- aligned the normal-navigation CPU usage distribution with the Jetson Nano pilot run ($5.87 \pm 4.15\%$);

- aligned normal-navigation LiDAR summary features with the successful LD08 pilot feature tail, including `min_lidar_distance_m` (0.3263 ± 0.0005 m) and `mean_lidar_distance_m` (2.6778 ± 0.0214 m);
- retained separate synthetic scenario profiles for anomalies that were not directly recorded in the pilot stage.

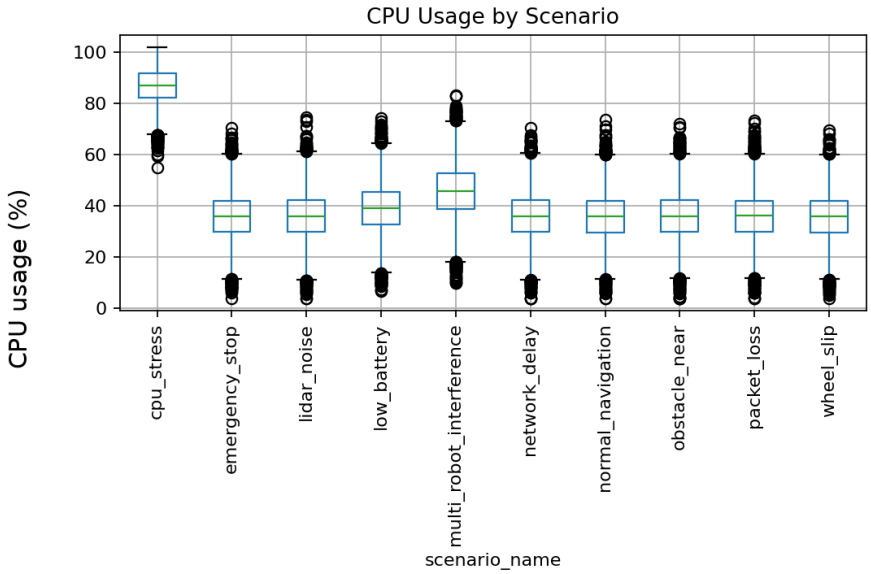


Figure 6. Box-plot comparison of CPU usage distributions by scenario. The x-axis lists scenario names, and the y-axis reports CPU usage as a percentage. Boxes indicate interquartile ranges, green lines indicate medians, whiskers summarize the non-outlier spread, and black circles indicate outlier observations.

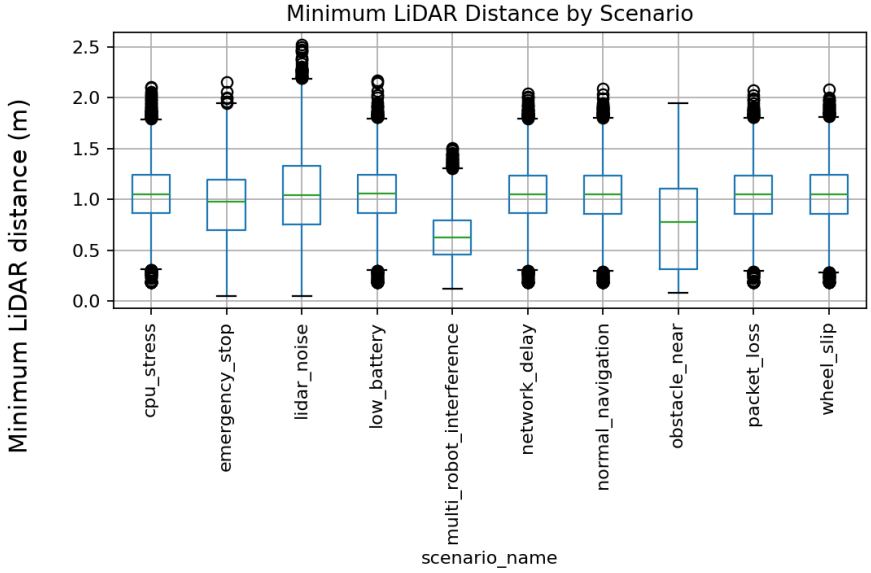


Figure 7. Box-plot comparison of engineered minimum LiDAR distance distributions by scenario. The x-axis lists scenario names, and the y-axis reports the minimum LiDAR distance in meters. Boxes indicate interquartile ranges, green lines indicate medians, whiskers summarize the non-outlier spread, and black circles indicate outlier observations.

5.3. Machine Learning Suitability Baseline

As an initial modeling check, we trained Random Forest baselines on the tabular benchmark using scikit-learn [36,37]. The split was configured by run identifier: for each

scenario, the final two runs were held out for testing and the remaining eight runs were used for training. This produced 96,000 training rows and 24,000 test rows without sharing run IDs across splits.

We evaluated performance on three baseline tasks:

1. **Binary Anomaly Detection:** Identifying nominal operations (`normal_navigation`) versus all anomaly scenarios. The run-based Random Forest baseline achieved accuracy, precision, recall, F1-score, AUC-ROC, and balanced accuracy of 1.000 on the held-out runs.
2. **Multi-Class Scenario Classification:** Classifying observations into the ten individual scenario classes. The Random Forest baseline achieved ten-class accuracy of 0.9286, macro-F1 of 0.9267, and weighted-F1 of 0.9267. Figure 8 shows the multiclass confusion matrix.
3. **Regression Severity Prediction:** Predicting the continuous anomaly severity parameter. The Random Forest regressor achieved MSE of 0.0081, MAE of 0.0531, and R^2 of 0.8406 on the held-out runs.

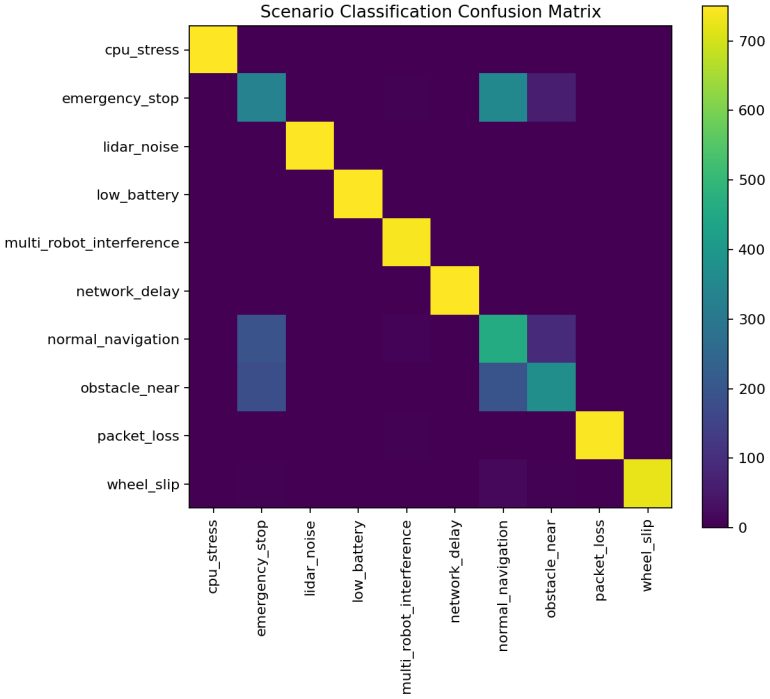


Figure 8. Multi-class Random Forest confusion matrix for held-out runs. Rows represent true scenario labels, columns represent predicted labels, diagonal cells represent correct classifications, and off-diagonal cells show scenario confusions.

Figure 8 shows where the multi-class model separates scenarios and where it confuses related operational patterns. Strong diagonal values indicate that most scenarios have distinct feature signatures, while off-diagonal entries reveal overlap between neighboring conditions, such as scenarios that affect braking, local obstacle proximity, or motion consistency in similar ways.

Figure 9 shows that the Random Forest model does not rely on a single variable alone. The ranking instead highlights the combined predictive value of LiDAR distance metrics, edge-compute load variables, and communication-related indicators. This supports the use of the dataset as a multi-channel benchmark rather than a one-signal thresholding example.

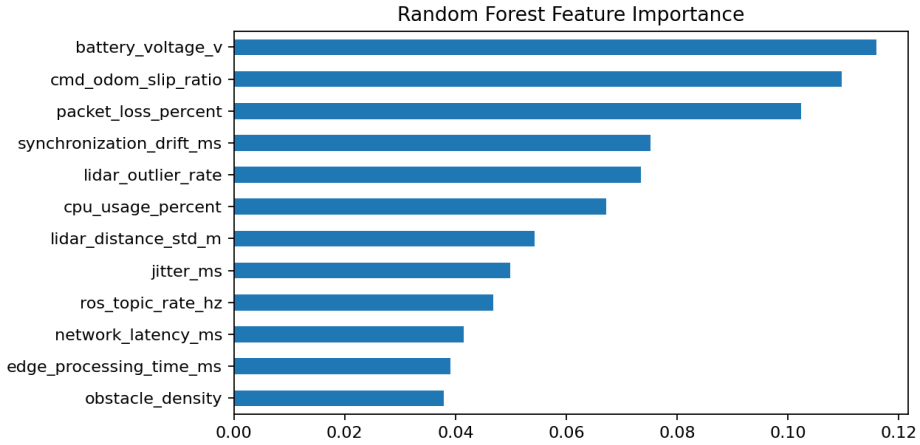


Figure 9. Random Forest feature-importance ranking for the tabular benchmark. Higher values indicate stronger contribution to the trained model's split decisions, with LiDAR, edge-compute, and network variables contributing to scenario separation.

These baseline results indicate that the dataset carries strong predictive signals for binary anomaly detection and useful, though not perfect, separation among individual scenarios. The lower F1-scores for some neighboring classes, such as `obstacle_near` and `emergency_stop`, are also informative because they leave room for model comparison rather than making the benchmark trivially solved. The high Random Forest scores should not be interpreted as evidence that a threshold-only or manually rule-based detector is sufficient for all tasks. Simple rules may detect isolated cases such as very high CPU usage or very short LiDAR distance, but they are likely to be brittle when anomalies overlap across channels, when operating ranges shift, or when multi-class distinctions require feature interactions. The Random Forest baseline benefits from supervised labels and multivariate split decisions; therefore, it is reported as a reproducibility and suitability check, not as a claim that operational anomaly detection is solved. The validation package can also support future gradient-boosted baselines and post-hoc interpretability workflows, such as XGBoost and SHAP-style explanations [38,39].

6. Usage Notes

The dataset can support several robotic-IoT and edge-computing tasks:

1. **Edge ML Prototyping:** Researchers can train compact classifiers intended for edge SBCs such as the Jetson Nano or Raspberry Pi.
2. **Evaluating Sensor Fusion:** The synchronized timelines link kinematic, LiDAR, compute, and network metrics, allowing sensor-fusion methods to be tested on a shared tabular record.
3. **Benchmarking ML Models:** The dataset provides a structured testbed for comparing tree ensembles, deep neural networks, and anomaly-detection algorithms under hardware and network constraints.
4. **Simulated Anomaly Testing:** The nine active failure scenarios allow models to be evaluated under specific anomalous events.

The Zenodo repository also includes example Jupyter notebooks (`notebooks/`) and validation scripts. These files show how to load the primary dataset, visualize target metrics across scenarios, and evaluate baseline ML models.

7. Limitations

Several limitations should be considered when using the dataset. They are summarized in Table 9 and discussed below.

Table 9. Identified study limitations and corresponding academic reporting mitigation strategies.

Identified Dataset Limitation	Methodological Impact	Academic Reporting Mitigation & Transparency
Partial Synthetic Benchmark	The full 10-scenario sequences are built from scenario-specific rules.	Accurately titled as a "pilot-real-calibrated robotic IoT benchmark" across all sections.
No Synchronized Master Bag	Complete cross-device messages were logged in distinct, non-synchronized rosbag streams.	Converted discrete parameters to a unified timeline using statistical distribution calibration.
Separated Edge Data Collection	NVIDIA Jetson Nano edge performance was logged in separate normal-navigation pilot runs rather than synchronously across all anomaly scenarios.	Calibrated the edge metrics column of S01 directly with real Jetson Nano `tegrastats` runs.
Simplified Indoor Environments	Geometric indoor boundaries use static obstacle placements during runs.	Shared the full logging, processing, and calibration suite in the `/ros` folder for local setups.

The main experimental limitation is that the package is not a fully unified physical-hardware recording. Collecting and labeling 100 long-duration runs under complex anomalies on real physical bases presents hardware risks, especially for close-obstacle, emergency-stop, low-battery, and wheel-slip conditions. We therefore used individual physical runs to calibrate a simulated benchmark. This gives researchers a controlled file for prototyping, but it does not replace validation on physical robots.

The absence of a synchronized master rosbag also affects interpretation. The ROS pilot runs, LD08 LiDAR verification, and edge-compute logging were collected as separate evidence streams rather than as a single fully synchronized multi-device campaign. The submitted benchmark therefore uses calibration and timestamped tabular synthesis to provide a unified analytical file, but it should not be treated as a raw reproduction of every cross-device packet and ROS message.

The Jetson Nano measurements provide empirical normal navigation levels for CPU usage, RAM usage, device temperature, GPU load, and edge-processing time. However, these edge measurements were not collected synchronously across all nine anomaly scenarios. Consequently, the calibrated edge columns are appropriate for evaluating tabular ML pipelines and nominal edge operating ranges, but not for drawing strong conclusions about Jetson behavior under every physical anomaly.

Finally, the indoor environment model uses simplified geometric assumptions and static obstacle placements. This choice improves reproducibility and makes the benchmark easy to inspect, but it limits direct claims about complex dynamic environments, human-robot interaction, lighting-dependent perception, and high-frequency control-loop behavior. The provided ROS scripts and future collection plan are included so that the benchmark can be extended toward more diverse physical deployments.

8. Future Real-World Data Collection Plan

To build on this pilot-calibrated dataset, we propose a three-stage plan (Table 10) for moving from this calibrated benchmark toward a fully physical, multi-device experimental campaign.

The physical deployment steps include:

1. **Deploying Unified Logging:** The logging tools provided in the `ros/` folder will be used so that robotics, network, and edge sensors write to a unified 10 Hz timeline.
2. **Active Anomaly Injection:** Automated tools will be added to inject network delays and packet drops, allowing communication failures to be recorded on real hardware under controlled conditions.

3. **Controlled Failure Runs:** Physical obstructions, motor slips, and low-battery events will be executed in a controlled laboratory environment.
4. **Publishing Real-World Telemetry:** The resulting fully physical, high-frequency dataset will be published under a new Zenodo DOI, allowing researchers to evaluate how models trained on this benchmark transfer to physical installations.

Table 10. Roadmap detailing research stages for full physical experimental replication and validation.

Research Phase	Device Setup	Operational Focus	Expected Deliverables
Phase 1: Minimum Viable Setup	Single TurtleBot3 Burger, Jetson Nano edge node, Raspberry Pi network logger.	Calibrate baseline configurations, verify LiDAR driver, log edge workloads under nominal routes.	Real ROS bag telemetry containing `/scan`, `/odom`, and `/cmd_vel`. Synchronized edge stats.
Phase 2: Dual-Robot Setup	Two namespaced TurtleBot3 Burgers, Jetson Nano, and multiple Pi network loggers.	Collect multi-robot interference and network contention samples under overlapping path designs.	Namespaced ROS bags and synchronized `merged_dataset_real`.
Phase 3: Extended Edge-IoT Fleet	Multiple mobile bases, Jetson Nano, three laptops, and up to ten Pi network monitors.	Evaluate distributed network and computing workloads, with multi-node telemetry.	Extended physical dataset for evaluating model transferability under complex deployment loads.

9. Conclusions

This paper presented a pilot-real-calibrated indoor robotic IoT benchmark dataset for edge-assisted mobile robot navigation and anomaly detection. The package connects a controlled tabular machine-learning benchmark with physical pilot evidence from a TurtleBot3 Burger, LD08 LiDAR bringup, and NVIDIA Jetson Nano telemetry. Throughout the manuscript, physical pilot evidence is separated from the calibrated synthetic benchmark rows.

The calibration pipeline aligned normal-navigation LiDAR and edge-compute distributions with empirical pilot statistics while retaining ten balanced scenario profiles and 120,000 tabular rows for reproducible experiments. Run-based Random Forest validation showed strong binary anomaly-detection performance and ten-class scenario accuracy of 0.9286. These results indicate that the dataset is suitable for baseline benchmarking while still preserving nontrivial multiclass distinctions. The Zenodo package (https://doi.org/10.5281/zenodo.20346694) provides documentation, metadata, validation scripts, ROS collection utilities, and a staged plan for future physical expansion.

At the same time, the dataset is not intended to replace raw SLAM, visual perception, formal safety certification, or high-frequency control datasets. It should not be used as the sole evidence base for claims about real-world robot transfer, dense trajectory reconstruction, visual anomaly detection, or hardware behavior under unobserved physical failures. Its intended scope is reproducible tabular prototyping for edge-assisted mobile-robot anomaly detection, with physical pilot evidence used to calibrate nominal operating ranges.

Funding: This research received no external grant funding. The article processing charge (APC) for this manuscript was supported by Istanbul Topkapi University. No grant number is applicable.

Institutional Review Board Statement: Not applicable. This study does not involve human subjects or animals.

Informed Consent Statement: Not applicable.

Data Availability Statement: The dataset presented in this study is openly available in Zenodo under the Creative Commons Attribution 4.0 International license [23].

Conflicts of Interest: The author declares no conflicts of interest. The APC supporter had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

References

1. Macenski, S.; Foote, T.; Gerkey, B.; Lalancette, C.; Woodall, W. Robot Operating System 2: Design, Architecture, and Uses in the Wild. *Sci. Robot.* **2022**, *7*, eabm6074. [CrossRef] [PubMed]
2. Macenski, S.; Moore, T.; Lu, D.V.; Merzlyakov, A.; Ferguson, M. From the Desks of ROS Maintainers: A Survey of Modern and Capable Mobile Robotics Algorithms in the Robot Operating System 2. *Robot. Auton. Syst.* **2023**, *168*, 104493. [CrossRef]
3. Albonico, M.; Djordjevic, M.; Hamer, E.; Malavolta, I. Software Engineering Research on the Robot Operating System: A Systematic Mapping Study. *J. Syst. Softw.* **2023**, *197*, 111574. [CrossRef]
4. Puccetti, T.; Nardi, S.; Cinquilli, C.; Zoppi, T.; Ceccarelli, A. ROSPaCe: Intrusion Detection Dataset for a ROS2-Based Cyber-Physical System and IoT Networks. *Sci. Data* **2024**, *11*, 481. [CrossRef] [PubMed]
5. Trilles, S.; Hammad, S.S.; Iskandaryan, D. Anomaly Detection Based on Artificial Intelligence of Things: A Systematic Literature Mapping. *Internet Things* **2024**, *25*, 101063. [CrossRef]
6. Ghaffari, A.; Jelodari, N.; Pouralish, S.; Derakhshanfard, N.; Arasteh, B. Securing Internet of Things Using Machine and Deep Learning Methods: A Survey. *Clust. Comput.* **2024**, *27*, 9065–9089. [CrossRef]
7. Nkoom, M.; Hounsinou, S.G.; Crosby, G.V. Securing the Internet of Robotic Things: A Comprehensive Review on Machine Learning-Based Intrusion Detection. *J. Cyber Secur. Technol.* **2024**, 1–50. [CrossRef]
8. Sharma, M.; Tomar, A.; Hazra, A. Edge Computing for Industry 5.0: Fundamental, Applications and Research Challenges. *IEEE Internet Things J.* **2024**, *11*, 19070–19093. [CrossRef]
9. Chatterjee, A.; Ahmed, B.S. IoT Anomaly Detection Methods and Applications: A Survey. *Internet Things* **2022**, *19*, 100568. [CrossRef]
10. Pang, G.; Shen, C.; Cao, L.; van den Hengel, A. Deep Learning for Anomaly Detection: A Review. *ACM Comput. Surv.* **2021**, *54*, 1–38. [CrossRef]
11. Ruff, L.; Kauffmann, J.R.; Vandermeulen, R.A.; Montavon, G.; Samek, W.; Kloft, M.; Dietterich, T.G.; Müller, K.R. A Unifying Review of Deep and Shallow Anomaly Detection. *Proc. IEEE* **2021**, *109*, 756–795. [CrossRef]
12. Brockmann, J.T.; Rudolph, M.; Rosenhahn, B.; Wandt, B. The voraus-AD Dataset for Anomaly Detection in Robot Applications. *IEEE Trans. Robot.* **2023**, *40*, 438–451. [CrossRef]
13. Mantegazza, D.; Xhyra, A.; Gambardella, L.M.; Giusti, A.; Guzzi, J. Hazards & Robots: A Dataset for Visual Anomaly Detection in Robotics. *Data Brief* **2023**, *48*, 109264. [CrossRef] [PubMed]
14. Değirmenci, E.; Kırca, Y.S.; Özçelik, İ.; Yazıcı, A. ROSIDS23: Network Intrusion Detection Dataset for Robot Operating System. *Data Brief* **2023**, *51*, 109739. [CrossRef] [PubMed]
15. Kayan, H.; Heartfield, R.; Rana, O.; Burnap, P.; Perera, C. CASPER: Context-Aware IoT Anomaly Detection System for Industrial Robotic Arms. *ACM Trans. Internet Things* **2024**, *5*, 1–36. [CrossRef]
16. Kayan, H. Anomaly Detection in Industrial Robotic Arms Using Edge-Based IoT Systems. Ph.D. Thesis, Cardiff University, Cardiff, Wales, 2025. Available online: https://orca.cardiff.ac.uk/id/eprint/179739/ (accessed on 4 June 2026).
17. Kayan, H.; Heartfield, R.; Rana, O.; Burnap, P.; Perera, C. Real-Time Anomaly Detection for Industrial Robotic Arms Using Edge Computing. *IEEE Internet Things J.* **2025**, *12*, 29696–29712. [CrossRef]
18. Reis, M. Lightweight Signal Processing and Edge AI for Real-Time Anomaly Detection in IoT Sensor Networks. *Sensors* **2025**, *25*, 6629. [CrossRef] [PubMed]
19. Helmberger, M.; Morin, K.; Berner, B.; Kumar, N.; Cioffi, G.; Scaramuzza, D. The Hilti SLAM Challenge Dataset. *IEEE Robot. Autom. Lett.* **2023**, *7*, 7518–7525. [CrossRef]
20. Zhang, L.; Helmberger, M.; Fu, L.F.T.; Wisth, D.; Camurri, M.; Scaramuzza, D.; Fallon, M. Hilti-Oxford Dataset: A Millimeter-Accurate Benchmark for Simultaneous Localization and Mapping. *IEEE Robot. Autom. Lett.* **2023**, *8*, 408–415. [CrossRef]
21. Wozniak, P.; Krzeszowski, T.; Kwolek, B. Multi-Domain Indoor Dataset for Visual Place Recognition and Anomaly Detection by Mobile Robots. *Sci. Data* **2025**, *12*, 817. [CrossRef] [PubMed]
22. Lin, S.; Chen, Z.; Jia, X.; Wang, Y.; Wang, C.; Zhang, S.; Ding, X.; Du, B.; Ding, W.; Liu, H.P. A Visual Dataset for Anomaly Detection in Self-Driving Laboratories. *Sci. Data* **2025**, *12*, 1787. [CrossRef] [PubMed]
23. Aggul, B. A Pilot-Real-Calibrated Indoor Robotic IoT Benchmark Dataset for Edge-Assisted Mobile Robot Navigation and Anomaly Detection (v1.0.0). Zenodo. 2026. Available online: https://zenodo.org/records/20346694 (accessed on 4 June 2026).
24. Wilkinson, M.D.; Dumontier, M.; Aalbersberg, I.J.J.; Appleton, G.; Axton, M.; Baak, A.; Blomberg, N.; Boiten, J.W.; da Silva Santos, L.B.; Bourne, P.E.; et al. The FAIR Guiding Principles for Scientific Data Management and Stewardship. *Sci. Data* **2016**, *3*, 160018. [CrossRef] [PubMed]

25. Lee, D.; Jung, M.; Yang, W.; Kim, A. LiDAR Odometry Survey: Recent Advancements and Remaining Challenges. *Intell. Serv. Robot.* **2024**, *17*, 95–118. [CrossRef]
26. Zhu, J.; Li, H.; Zhang, T. Camera, LiDAR, and IMU Based Multi-Sensor Fusion SLAM: A Survey. *Tsinghua Sci. Technol.* **2024**, *29*, 415–429. [CrossRef]
27. Fan, Z.; Zhang, L.; Wang, X.; Shen, Y.; Deng, F. LiDAR, IMU, and Camera Fusion for Simultaneous Localization and Mapping: A Systematic Review. *Artif. Intell. Rev.* **2025**, *58*, 174. [CrossRef]
28. Li, Y.; An, J.; He, N.; Li, Y.; Han, Z.; Chen, Z.; Qu, Y. A Review of Simultaneous Localization and Mapping Algorithms Based on Lidar. *World Electr. Veh. J.* **2025**, *16*, 56. [CrossRef]
29. Roch, J.; Fayyad, J.; Najjaran, H. DOPESLAM: High-Precision ROS-Based Semantic 3D SLAM in a Dynamic Environment. *Sensors* **2023**, *23*, 4364. [CrossRef] [PubMed]
30. ROBOTIS. TurtleBot3 Features and Specifications. 2026. Available online: https://emanual.robotis.com/docs/en/platform/turtlebot3/features/#specifications (accessed on 4 June 2026).
31. NVIDIA. Jetson Nano Developer Kit. 2026. Available online: https://developer.nvidia.com/embedded/jetson-nano-developer-kit (accessed on 4 June 2026).
32. Akli, I.; Abdous, K.; Abderrahmane, S. Hardware in the Loop Simulation for Robot Navigation with RFID. In *Proceedings of the 2022 7th International Conference on Image and Signal Processing and Their Applications (ISPA), Mostaganem, Algeria, 8–9 May 2022*; IEEE: New York, NY, USA, 2022; pp. 1–6. Available online: https://ieeexplore.ieee.org/document/9786321 (accessed on 4 June 2026)
33. Tiboni, G.; Arndt, K.; Kyrki, V. DROPO: Sim-to-Real Transfer with Offline Domain Randomization. *Robot. Auton. Syst.* **2023**, *166*, 104432. [CrossRef]
34. Allka, X.; Ferrer-Cid, P.; Barceló-Ordinas, J.M.; Garcia-Vidal, J.; Avila-Torrado, A. ESOD: An Edge Streaming Data Outlier Detection Framework for IoT Platforms. *IEEE Internet Things J.* **2025**, *12*, 35140–35149. [CrossRef]
35. *ISO 8601-1:2019*; Date and Time–Representations for Information Interchange–Part 1: Basic Rules. International Organization for Standardization: Geneva, Switzerland, 2019.
36. Breiman, L. Random Forests. *Mach. Learn.* **2001**, *45*, 5–32. [CrossRef]
37. Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; et al. Scikit-learn: Machine Learning in Python. *J. Mach. Learn. Res.* **2011**, *12*, 2825–2830. Available online: http://jmlr.org/papers/v12/pedregosa11a.html (accessed on 4 June 2026).
38. Chen, T.; Guestrin, C. XGBoost: A Scalable Tree Boosting System. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, New York, NY, USA, 13–17 August 2016; pp. 785–794. [CrossRef]
39. Lundberg, S.M.; Lee, S.I. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Red Hook, NY, USA, 2017; Volume 30. Available online: https://proceedings.neurips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf (accessed on 4 June 2026).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.